

Çağla Ökmen

Homework 5 Dokümantasyonu:

Bu dokümantasyonda, TPS problem yapısı değiştirilerek "Bölgesel TSP" (Regional TSP) senaryosu uygulanmıştır. Bu senaryoda 10 farklı merkez belirlenmiş, bu merkezlerin 200 metre yarıçapındaki komşuları bir "bölge" olarak kabul edilmiştir. Amaç, her bölgeden sadece bir nokta seçerek toplam mesafeyi minimize eden en uygun rotayı bulmaktır.

Kod Açıklaması:

1. Main.py

```
TPS > main.py > ...
1  import osmnx as ox
2  import networkx as nx
3  import random
4  import time
5  import pandas as pd
6  import numpy as np
7  import os
8
9  from ortools_metot import tps_ortool
10 from greedy_intertion import tps_insertion
11 from genetic import tps_genetic
12 from folium_map import draw_map
13 from comparison_graphs import comparison
14
15 random.seed(9)
16 np.random.seed(9)
```

Gerekli kütüphaneler ve diğer dosyalardaki fonksiyonlar import edildi. Seed değeri ayarlandı.

```
17 def neighborhoods(point_node, G, radius):
18     lengths = nx.single_source_dijkstra_path_length(G, point_node, cutoff=radius, weight="length")
19     return list(lengths.keys())
```

Önceki ödevlerden farklı olarak Node'lar kümeler halinde gruplandırılmıştır. neighborhoods fonksiyonu ile harita üzerinde seçilen merkez noktalara belirli bir yarıçap mesafedeki komşu Node'lar bulunarak bir bölge tanımlanmıştır.

```
21 def generate_graph(G, size):
22     nodes = list(G.nodes())
23     N = nx.Graph()
24     point_nodes = []
25
26     # Bağlantı kontrolü eklendi Eklendi
27     # Bağlantısı Olmayan Nodeları Alma
28     while len(point_nodes) < size:
29         candidate = random.choice(nodes)
30         # Eger daha önce seçilmişse geç
31         if candidate in point_nodes:
32             continue
33         ok = True
34         for n in point_nodes:
35             if not (nx.has_path(G, candidate, n)):
36                 ok = False
37                 break
38         if ok:
39             point_nodes.append(candidate)
40
41     # Node içeriği {'y': 40.1961554, 'x': 28.9545733, 'street_count': 3}
42     # Rastgele alınan Node larla yeni graf oluşturun
43     for i in range(len(point_nodes)):
44         N.add_node(point_nodes[i], x = G.nodes[point_nodes[i]]['x'], y = G.nodes[point_nodes[i]]['y'])
45     return N
```

generate_graph_structure fonksiyonunda ise rastgele seçilen Node ların birbirleriyle bağlantısı olup olmadığı kontrol edilerek N grafi oluşturuldu. Harita indirme main tarafına eklendi ve nodlar arası yol kısımları kaldırıldı.

```

47 # Mesafe matrisini önceden hesaplayarak işlem süresini azaltma
48 def precompute_distances(G, all_candidate_nodes):
49     dist_matrix = {}
50     print(f"Mesafe Matrisi Önbelleğe Alınıyor ({len(all_candidate_nodes)} düğüm)...")
51
52     for source in all_candidate_nodes:
53         lengths = nx.single_source_dijkstra_path_length(G, source, weight="length")
54         dist_matrix[source] = lengths
55
56     return dist_matrix
57

```

Bölgesel yaklaşımda çok fazla aday node olduğu için, algoritmalar çalışmadan önce precompute_distances fonksiyonu ile gerekli tüm Node'lar arası mesafeler hesaplanıp bir sözlükte tutuldu. Bu sayede algoritmaların çalışma süreleri optimize edildi.

```

59 # Ana program
60 if __name__ == '__main__':
61     place = "Nilüfer, Bursa, Turkey"
62     map_file = "bursa_nilufer_static.graphml" # Sabit Harita Dosyası
63
64     # Harita yükleme
65     if os.path.exists(map_file):
66         print(f"Sabit harita '{map_file}' dosyasından yükleniyor...")
67         G = ox.load_graphml(map_file)
68     else:
69         print("Harita internetten indiriliyor ve sabitleniyor (Sadece 1 kez)...")
70         G = ox.graph_from_place(place, network_type='drive')
71         G = ox.truncate.largest_component(G, strongly=True)
72
73     # Haritayı kaydet
74     ox.save_graphml(G, map_file)
75     print(f"Harita '{map_file}' olarak kaydedildi.")
76
77     print("Harita hazır.")
78
79     # Sonuçları tutacak listeler
80     heur_sums, heur_times = [], []
81     ortool_sums, ortool_times = [], []
82     gen_sums, gen_times = [], []
83

```

"Nilüfer, Bursa" haritası indirildi. OpenStreetMap (OSM) canlı bir harita olduğu için yollar değişebilmektedir. Farklı ortamlarda daha tutarlı sonuçlar almak için harita dosyası indirilir.

```

84 for i in range(30):
85     # seed ayarla
86     random.seed(i + 1)
87     np.random.seed(i + 1)
88     print(f"\n{i + 1}.Topoloji Çalışıyor...")
89
90     N = generate_graph(G, 10)
91
92     search_radius = 200
93     regions_dict = {}
94     regions_info = {}
95     all_involved_nodes = set()
96
97     # bölgeleri oluşturma
98     for center in N.nodes():
99         neighbors = neighborhoods(center, G, search_radius)
100         if not neighbors:
101             neighbors = [center]
102         regions_dict[center] = neighbors
103         regions_info[f"R{center}"] = {"center": center, "nodes": neighbors}
104         all_involved_nodes.update(neighbors)
105
106     # Mesafeleri önceden hesapla
107     t0 = time.time()
108     dist_matrix = precompute_distances(G, list(all_involved_nodes))
109

```

30 farklı topoloji için döngü oluşturuldu. Her topoloji de rastgele seçilen merkezler için 200 metre yarıçapındaki komşular bulunarak regions_dict sözlüğüne kaydedildi.

```

106     # Mesafeleri önceden hesapla
107     t0 = time.time()
108     dist_matrix = precompute_distances(G, list(all_involved_nodes))
109
110     # heuristic yontemi
111     start_heur = time.time()
112     heuristic_sum, heuristic_path = tps_insertion(N, dist_matrix, regions_dict)
113     heur_times.append(time.time() - start_heur)
114     heur_sums.append(heuristic_sum / 1000)
115
116     # Or-tool yontemi
117     start_ortool = time.time()
118     ortool_sum, ortool_path = tps_ortool(dist_matrix, regions_dict)
119     ortool_times.append(time.time() - start_ortool)
120     ortool_sums.append(ortool_sum / 1000)
121
122     # Genetik yontemi
123     start_gen = time.time()
124     gen_path, gen_fit = tps_genetic(dist_matrix, regions_dict, pop_size=50, mutation_rate=0.1, generations=20)
125     gen_times.append(time.time() - start_gen)
126     gen_sums.append(gen_fit / 1000)

```

Sırasıyla tps_insertion, tps_ortool ve tps_genetic fonksiyonları çalıştırıldı.

```

128     # 5 topoloji de bir harita çizdir
129     if (i + 1) % 5 == 0:
130         print(f"> {i+1}. Topoloji için Haritalar oluşturuluyor...")
131
132         current_lengths = {
133             'Heuristic': heuristic_sum,
134             'OR-Tools': ortool_sum,
135             'Genetic': gen_fit
136         }
137         paths_combined = {
138             'Heuristic': heuristic_path,
139             'OR-Tools': ortool_path,
140             'Genetic': gen_path
141         }
142         draw_map(G, regions_info, paths_combined, current_lengths, search_radius, f"Topoloji_{i+1}_Karsilastirma")
143
144         # Ayırı ayrı haritalar
145         if heuristic_path:
146             draw_map(G, regions_info, {'Heuristic': heuristic_path}, current_lengths, search_radius, f"Topoloji_{i+1}_Heuristic")
147         if ortool_path:
148             draw_map(G, regions_info, {'OR-Tools': ortool_path}, current_lengths, search_radius, f"Topoloji_{i+1}_ORTools")
149         if gen_path:
150             draw_map(G, regions_info, {'Genetic': gen_path}, current_lengths, search_radius, f"Topoloji_{i+1}_Genetic")
151
152         print(f"Haritalar kaydedildi (T{i+1})")
153     # Pandas ile Sonuçları kaydetme
154     df = pd.DataFrame([
155         'Topoloji_No': range(1, len(heur_sums) + 1),
156         'Heuristic_Sum_km': heur_sums,
157         'Heuristic_Time_s': heur_times,
158         'OR_Tools_Sum_km': ortool_sums,
159         'OR_Tools_Time_s': ortool_times,
160         'Genetic_Sum_km': gen_sums,
161         'Genetic_Time_s': gen_times
162     ])
163
164     df.to_csv("regional_tsp_results.csv", index=False)
165     print("\nVeriler 'regional_tsp_results.csv' dosyasına kaydedildi.")
166     comparison(heur_sums, heur_times, ortool_sums, ortool_times, gen_sums, gen_times)

```

Her yöntemin çalışma süresi ve toplam yol uzunluğu kaydedildi. Her 5 topolojide bir, draw_map fonksiyonu ile bölgeleri ve seçilen yolları gösteren haritalar oluşturuldu. 30 topolojinin tüm sonuçları regional_tsp_results.csv dosyasına kaydedildi. Son olarak comparison fonksiyonu ile yöntemlerin performans karşılaştırma grafikleri çizdirildi.

2. Greedy_intertion.py

```
4 def best_node_add(dist_matrix, path, unvisited_regions, regions):
5     sm_dis = float('inf')
6     best_region = None
7     best_node_in_region = None
8     insert_index = -1
9
10    for region_center in unvisited_regions:
11        candidates = regions[region_center]
12        for candidate_node in candidates:
13            # Mevcut yoldaki aralıklara ekleme maliyeti
14            for i in range(len(path) - 1):
15                n1 = path[i]
16                n2 = path[i + 1]
17
18                try:
19                    d1 = dist_matrix[n1][candidate_node]
20                    d2 = dist_matrix[candidate_node][n2]
21                    base_dist = dist_matrix[n1][n2]
22
23                    distance = d1 + d2 - base_dist
24
25                    if distance < sm_dis:
26                        sm_dis = distance
27                        insert_index = i + 1
28                        best_node_in_region = candidate_node
29                        best_region = region_center
30                except KeyError:
31                    continue
32
33    if best_region is not None:
34        path.insert(insert_index, best_node_in_region)
35    return path, best_region
36    return path, None
```

best_node_add fonksiyonu, henüz ziyaret edilmemiş her bir bölge için o bölgeye ait tüm aday nodeları tek tek değerlendirir. Değerlendirmeden sonra, maliyet artışı en düşük olan Node ve ait olduğu bölge seçilerek mevcut tura eklenir.

```
37
38 # heuristic algoritma uygulaması
39 def tps_insertion(N, dist_matrix, regions):
40     unvisited_regions = list(regions.keys())
41
42     # Random 3 Node belirle
43     init_centers = random.sample(unvisited_regions, 3)
44     path = [random.choice(regions[c]) for c in init_centers]
45
46     for c in init_centers:
47         unvisited_regions.remove(c)
48
49     path.append(path[0])
50
51     # Tüm Nodelar ziyaret edilene kadar çalıştır. En iyi node veren yolu günceller
52     while unvisited_regions:
53         path, visited_region = best_node_add(dist_matrix, path, unvisited_regions, regions)
54         if visited_region:
55             unvisited_regions.remove(visited_region)
56
57     # Toplam yolu hesapla
58     total_dist = 0
59     for i in range(len(path)-1):
60         total_dist += dist_matrix[path[i]][path[i+1]]
61
62     return total_dist, path
```

Rastgele seçilen 3 farklı bölgeden, rastgele birer Node seçilerek başlangıç rotası oluşturularak başlar. While döngüsü ile tüm bölgeler ziyaret edilene kadar best_node_add fonksiyonu çağrılır ve en uygun Nodelar tura eklenir. Son olarak elde edilen turun toplam mesafesi hesaplanarak döndürülür.

3. Ortools_metot.py

```
1  from ortools.constraint_solver import routing_enums_pb2
2  from ortools.constraint_solver import pywrapcp
3
4  # Ortool tam sayıya donuřturmesinden elde edilen
5  # kayıpların olamamsı için Toplam yolun tekrar hesaplanır
6  def sum_distance(dist_matrix, path):
7      total_dist = 0
8      for i in range(len(path) - 1):
9          try:
10             total_dist += dist_matrix[path[i]][path[i+1]]
11          except KeyError:
12             pass
13      return total_dist
14
15  def create_data_matrix(all_nodes, dist_matrix):
16      dis_matrix_list = []
17
18      for node1 in all_nodes:
19          row = []
20          for node2 in all_nodes:
21              if node1 == node2:
22                  row.append(0)
23              else:
24                  try:
25                      # Ortool tam sayı kabul ettiđi için donusturuldu
26                      val = int(dist_matrix[node1][node2] * 10000)
27                      row.append(val)
28                  except KeyError:
29                      # Yol yoksa çok büyük ceza
30                      row.append(100000000)
31              dis_matrix_list.append(row)
32
33      return dis_matrix_list
```

Rota bulunduktan sonra, ölçeklendirme kaynaklı hataları gidermek için sum_distance fonksiyonu ile orijinal mesafe matrisi kullanılarak rotanın gerçek uzunluđu hesaplanmıřtır. Create_data_matrix fonksiyonunda ondalıklı mesafeler hassasiyet kaybını önlemek için çarpım 10.000 e çıkarılarak tam sayıya dönüřtürülmüřtür.

```

34
35 # Or-tool ile TPS çözümü
36 def tps_ortool(dist_matrix, regions):
37     all_nodes = []
38     region_indices = []
39
40     current_idx = 0
41     # Her bölgedeki node'ların matristeki indekslerini grupla
42     for center, nodes in regions.items():
43         indices_in_this_region = []
44         for node in nodes:
45             all_nodes.append(node)
46             indices_in_this_region.append(current_idx)
47             current_idx += 1
48         region_indices.append(indices_in_this_region)
49
50     # Matris oluşturma
51     matris = create_data_matrix(all_nodes, dist_matrix)
52     if not matris: return 0, []
53
54     # Konum sayıları, mesafe matrisine ilişkin dizinlere karşılık gelir.
55     manager = pywrapcp.RoutingIndexManager(len(matris), 1, 0)
56     routing = pywrapcp.RoutingModel(manager)
57
58     # Mesafeyi geri çağırma
59     def distance_callback(from_index, to_index):
60         from_node = manager.IndexToNode(from_index)
61         to_node = manager.IndexToNode(to_index)
62         return matris[from_node][to_node]
63
64     transit_callback_index = routing.RegisterTransitCallback(distance_callback)
65     routing.SetArcCostEvaluatorOfAllVehicles(transit_callback_index)
66
67     # Her bölgeden sadece 1 node seçer
68     penalty = int(1e14)
69     for indices in region_indices:
70         routing.AddDisjunction(indices, penalty)
71
72     search_parameters = pywrapcp.DefaultRoutingSearchParameters()
73     search_parameters.first_solution_strategy = (routing_enums_pb2.FirstSolutionStrategy.PATH_CHEAPEST_ARC)
74
75     # Daha iyi sonuç için local search metodu
76     search_parameters.local_search_metaheuristic = (routing_enums_pb2.LocalSearchMetaheuristic.GUIDED_LOCAL_SEARCH)
77     search_parameters.time_limit.seconds = 5 # zaman sınırı
78
79     solution = routing.SolveWithParameters(search_parameters)

```

Tps_ortool fonksiyonu, bölge gruplarını ve mesafe matrisini alarak, her bölgeden sadece bir noktanın ziyaret edildiği en verimli rotayı bulmaktadır. Her bölgedeki node'lar eklendiği için çok sayıda Node vardır ve ortool yöntemi bölgesel TPS problemi için tasarlanmadığından daha iyi sonuçlar elde etmek amacıyla ortool'un sezgisel yöntemi parametrelere eklenmiştir. Bölgesel seçim yapabilmek için Node gruplarına yüksek bir ceza puanı atanarak algoritmanın her bölgeyi atlamadan sadece bir Node seçmesi zorunlu kılınmıştır. Algoritmanın sonsuz döngüye girmesini veya çok uzun süre çalışmasını engellemek için çözüm süresi 5 saniye ile sınırlandırılmıştır. Daha iyi sonuç elde etmek için süre artırılabilir.

```

81 # Sonuç
82 if solution:
83     index = routing.Start(0)
84     path = []
85
86     # Sadece node ID'lerini sırayla al
87     while not routing.IsEnd(index):
88         node_idx = manager.IndexToNode(index)
89         path.append(all_nodes[node_idx])
90         index = solution.Value(routing.NextVar(index))
91     path.append(all_nodes[manager.IndexToNode(index)])
92
93     real_total_dist = sum_distance(dist_matrix, path)
94     return real_total_dist, path
95 else:
96     return 0, []

```

Elde edilen çözüm rotası taranarak Nodelar listeye eklenir. Tam sayı dönüşümünden kaynaklı kayıplarını gidermek için toplam mesafe sum_distance ile orijinal matris üzerinden yeniden hesaplanarak döndürülür.

4. Genetic.py

```
TPS > genetic.py > tps_genetic
1  import random
2  import numpy as np
3
4  # Yolun Toplam uzunlugunu hesapla
5  def sum_distance(dist_matrix, path):
6      sum = 0
7      for i in range(len(path) - 1):
8          sum += dist_matrix[path[i]][path[i + 1]]
9
10     # son node ile ilk noduda hesaplama
11     sum += dist_matrix[path[-1]][path[0]]
12
13     return sum
```

Yol uzunluğu hesaplama fonksiyonu.

```
15  # Populasyon oluşturma
16  def create_population(pop_size, regions):
17      population = []
18      region_keys = list(regions.keys())
19      for _ in range(pop_size):
20          individual = []
21          random.shuffle(region_keys)
22          for r_key in region_keys:
23              selected_node = random.choice(regions[r_key])
24              individual.append((r_key, selected_node))
25          population.append(individual)
26      return population
27
28  # Populasyonu değerlendirme
29  def evaluate_population(population, dist_matrix):
30      fitness_scores = []
31      for indiv in population:
32          path_nodes = [node for _, node in indiv]
33          fitness_scores.append(sum_distance(dist_matrix, path_nodes))
34      return fitness_scores
35
36  # Seçim işlemi
37  def select(population, fitness_scores, num_best):
38      selected_indices = np.argsort(fitness_scores)[:num_best]
39      return [population[i] for i in selected_indices]
```

Create_population fonksiyonu ile başlangıç popülasyonu oluşturulurken iki seviyeli bir rassallık kullanılır: İlk olarak bölgelerin ziyaret sırasını belirlemek için bölge anahtarları karıştırılır, ardından her bir bölge için o bölgedeki aday Node'lardan biri rastgele seçilir. evaluate_population fonksiyonu ile her rotanın uygunluk değeri yani toplam mesafesi hesaplanır ve listeye kaydedilir. select fonksiyonu ise en iyi birey seçimini yapar.

```

40
41 # Çaprazlama işlemi
42 def crossover(parent1, parent2):
43     start, end = sorted(random.sample(range(len(parent1)), 2))
44     child = [None] * len(parent1)
45     child[start:end] = parent1[start:end]
46     current_regions = set(x[0] for x in child if x is not None)
47     pointer = 0
48     for i in parent2:
49         r_id, _ = i
50         if r_id not in current_regions:
51             while child[pointer] is not None: pointer += 1
52             child[pointer] = i
53             current_regions.add(r_id)
54     return child
55
56 # Mutasyon işlemi
57 def mutate(individual, mutation_rate, regions):
58     # bölge sıralaması değiştirme
59     if random.random() < mutation_rate:
60         i, j = random.sample(range(len(individual)), 2)
61         individual[i], individual[j] = individual[j], individual[i]
62     # bölge içerisinde farklı node ile değiştirme
63     if random.random() < mutation_rate:
64         idx = random.randint(0, len(individual) - 1)
65         r_id, _ = individual[idx]
66         new_node = random.choice(regions[r_id])
67         individual[idx] = (r_id, new_node)

```

Ebeveynlerden yeni bireyler üretmek için sıralı çaprazlama yöntemi kullanılır. parent1'den rastgele bir parça alınarak çocuğa aktarılır. Mutate fonksiyonu ile iki farklı mutasyon gerçekleştirilir. İlk olarak rotadaki iki bölgenin yerini değiştirerek ziyaret sırasını çeşitlendirir. İkinci olarak ise bölge sırasını koruyarak, seçilen bir bölgenin içerisindeki aktif Node başka bir komşu Node ile değiştirir. Bu sayede sadece rotayı değil durak noktalarını da iyileştirilmesi sağlanır.

```

68
69 # Genetik algoritma ana fonksiyonu
70 def tps_genetic(dist_matrix, regions, pop_size, mutation_rate, generations):
71     population = create_population(pop_size, regions)
72     best_path_nodes = None
73     best_fitness = float('inf')
74
75     elite_ratio = 0.1
76     elit_count = max(1, int(pop_size * elite_ratio))
77
78     for _ in range(generations):
79         fitness_scores = evaluate_population(population, dist_matrix)
80
81         min_idx = np.argmin(fitness_scores)
82         if fitness_scores[min_idx] < best_fitness:
83             best_fitness = fitness_scores[min_idx]
84             best_path_nodes = [node for _, node in population[min_idx]]
85
86         elite_indices = np.argsort(fitness_scores)[:elit_count]
87         new_population = [population[i].copy() for i in elite_indices]
88
89         selected = select(population, fitness_scores, pop_size // 2)
90
91         while len(new_population) < pop_size:
92             parent1, parent2 = random.sample(selected, 2)
93             child = crossover(parent1, parent2)
94             mutate(child, mutation_rate, regions)
95             new_population.append(child)
96         population = new_population
97
98     best_path_nodes.append(best_path_nodes[0])
99     return best_path_nodes, best_fitness

```

Genetik algoritmasının ana kodunda sırayla işlemleri gerçekleştirir.

```

TPS > folium.map.py > draw_map
1 import folium
2 from folium import plugins
3 import networkx as nx
4
5 # Folium Harita Çizimi
6 def draw_map(G, regions, paths_dict, lengths_dict, radius, name):
7     # Harita merkezi
8     centers = [r["center"] for r in regions.values()]
9     lat = sum(G.nodes[n]['y'] for n in centers) / len(centers)
10    lon = sum(G.nodes[n]['x'] for n in centers) / len(centers)
11    m = folium.Map(location=[lat, lon], zoom_start=13)
12
13    # Bölgeleri çiz
14    for r_id, r in regions.items():
15        c = r["center"]
16        folium.Circle(
17            location=[G.nodes[c]['y'], G.nodes[c]['x']],
18            radius=radius, color="gray", fill=True, fill_opacity=0.1,
19            popup=f"Bölge Merkezi: {c}"
20        ).add_to(m)
21
22    colors = {'Heuristic': 'blue', 'OR-Tools': 'red', 'Genetic': 'green'}
23
24    legend_items = []
25
26    # Her Yöntem İçin Yolları ve Noktaları Çiz
27    for method_name, path in paths_dict.items():
28        if not path: continue
29
30        c_code = colors.get(method_name, 'black')
31        dist_km = lengths_dict.get(method_name, 0) / 1000.0
32
33        legend_items.append(
34            f'&nbsp;<i style="background:{c_code}; width:10px; height:10px; display:inline-block;"></i>
35            f'<b>{method_name}</b> {dist_km:.2f} km')
36
37        # Seçilen Noktaları İşaretle
38        for node in path:
39            folium.CircleMarker(
40                location=[G.nodes[node]['y'], G.nodes[node]['x']],
41                radius=4, color=c_code, fill=True, fill_opacity=1.0,
42                popup=f"{method_name}: {node}"
43            ).add_to(m)
44
45        # Rotayı Çiz
46        route_coords = []
47        for i in range(len(path) - 1):
48            try:
49                sp = nx.shortest_path(G, path[i], path[i + 1], weight="length")
50                for node_id in sp:
51                    route_coords.append([G.nodes[node_id]['y'], G.nodes[node_id]['x']])
52            except nx.NetworkXNoPath:
53                continue
54
55        if route_coords:
56            folium.PolyLine(
57                route_coords,
58                color=c_code,
59                weight=3,
60                opacity=0.7,
61                # Yolum üzerine gelince mesafe yazar
62                tooltip=f"{method_name} Yolu ({dist_km:.2f} km)"
63            ).add_to(m)
64
65    # Dinamik Lejant (Bilgi Kutusu) Oluşturma
66    legend_html = f'''
67    <div style="position: fixed;
68    bottom: 50px; left: 50px; width: 220px; height: auto;
69    border: 2px solid grey; z-index: 9999; font-size: 14px;
70    background-color: white; opacity: 0.9; padding: 10px;">
71    <b>Sonuçlar (Toplam Mesafe):</b> <br>
72    {"<br>".join(legend_items)}
73    </div>
74    '''
75    m.get_root().html.add_child(folium.Element(legend_html))
76
77    filename = f"{name}.html"
78    m.save(filename)

```

draw_map fonksiyonunda, Folium kütüphanesi kullanılarak algoritmaların (Sezgisel, OR-Tools, Genetik) yollarını 4 farklı (hepsinin olduğu ve ayrı ayrı) harita ile oluşturulmuştur. Harita üzerinde bölgeler gri daireler ile gösterilmiştir.

```

TPS > comparison_graphs.py > ...
1  import matplotlib.pyplot as plt
2  import numpy as np
3
4  # Grafik Çizdirme
5  def comparison(heur_sums, heur_times, ortool_sums, ortool_times, gen_sums, gen_times):
6      num_topologies = len(heur_sums)
7      x_axis = range(1, num_topologies + 1)
8
9      # Yol Grafiği
10     plt.figure(figsize=(10, 6))
11     plt.plot(x_axis, heur_sums, label='Regional Heuristic', marker='o', linestyle='--')
12     plt.plot(x_axis, ortool_sums, label='Regional OR-Tools', marker='o', linestyle='-.')
13     plt.plot(x_axis, gen_sums, label='Regional Genetic', marker='o', markerfacecolor='none', linestyle=':')
14
15     plt.xlabel('Topoloji Numarası')
16     plt.ylabel('Toplam Mesafe')
17     plt.title('Bölgesel TSP Yol Karşılaştırması')
18     plt.legend()
19     plt.grid(True, alpha=0.3)
20
21     plt.savefig("karsilastirma_mesafe.png", dpi=150)
22
23     # Zaman Grafiği
24     plt.figure(figsize=(10, 6))
25     plt.plot(x_axis, heur_times, label='Regional Heuristic', marker='o', linestyle='--')
26     plt.plot(x_axis, ortool_times, label='Regional OR-Tools', marker='o', linestyle='-.')
27     plt.plot(x_axis, gen_times, label='Regional Genetic', marker='o', markerfacecolor='none', linestyle=':')
28
29     plt.yscale('symlog', linthresh=1e-4) # çok küçükleri görünür yapar
30
31     plt.xlabel('Topoloji Numarası')
32     plt.ylabel('Süre (saniye) - symlog')
33     plt.title('Algoritma Çalışma Süreleri')
34     plt.legend()
35     plt.xticks(x_axis)
36     plt.grid(True, which="both", linestyle='--', linewidth=0.5, alpha=0.7)
37
38     plt.savefig("karsilastirma_zaman.png", dpi=150)
39
40     # Ortool ile diğer metotların karşılaştırması (y=x)
41     plt.figure(figsize=(8, 8))
42
43     # Scatter noktaları
44     plt.scatter(ortool_sums, heur_sums, c='blue', label='Heuristic vs OR-Tools')
45     plt.scatter(ortool_sums, gen_sums, c='green', label='Genetic vs OR-Tools')
46
47     # y=x Referans Çizgisi
48     # Eksen limitlerini belirle
49     all_values = heur_sums + ortool_sums + gen_sums
50     min_val = min(all_values) * 0.95
51     max_val = max(all_values) * 1.05
52
53     plt.plot([min_val, max_val], [min_val, max_val], color='orange', linestyle='--', linewidth=2, label='y=x (Or-Tools)')
54
55     plt.xlim(min_val, max_val)
56     plt.ylim(min_val, max_val)
57     plt.xlabel('OR-Tools Maliyeti (km)')
58     plt.ylabel('Diğer Metotların Maliyeti (km)')
59     plt.title('OR-Tools Referansına Göre Yakınsama Analizi')
60     plt.legend()
61     plt.grid(True, alpha=0.4)
62     plt.tight_layout()
63     plt.savefig("karsilastirma_dagilim.png", dpi=150)
64

```

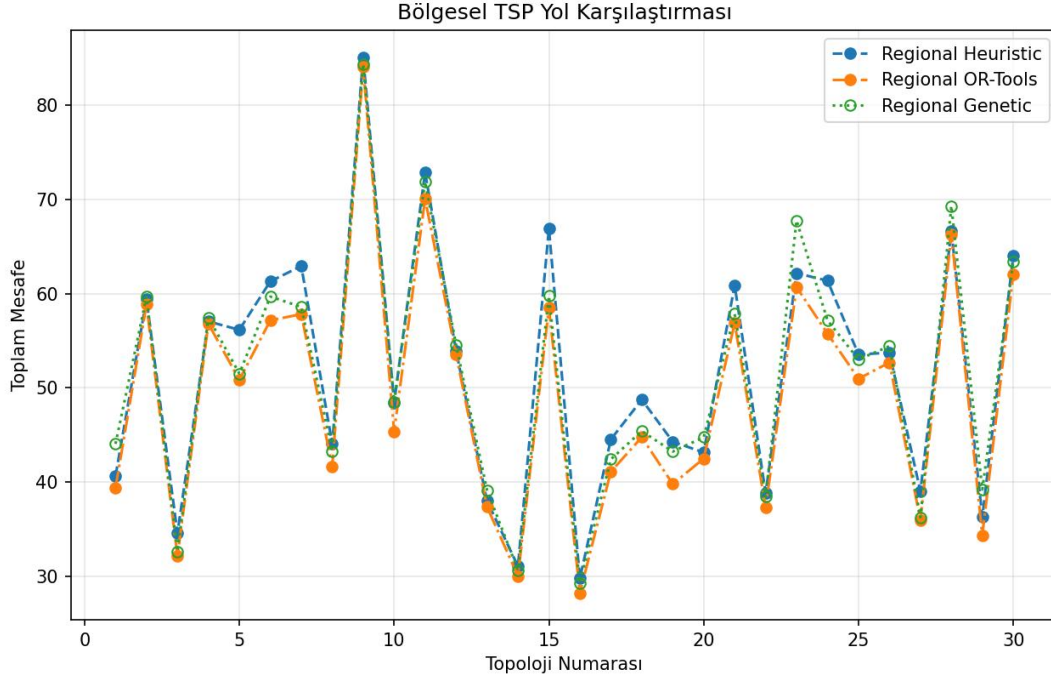
comparison fonksiyonu, 30 farklı topoloji üzerinde elde edilen mesafe ve süre verilerini görselleştirerek algoritmaların performans analizini yapar. Toplam yol maliyetleri ve logaritmik ölçekte çalışma süreleri çizgi grafikleriyle görselleştirilir. Ayrıca Sezgisel ve Genetik algoritmaların OR-Tools referansına göre sapmalarını ve yakınsama başarılarını analiz etmek amacıyla $y=x$ doğrusunu içeren bir dağılım grafiği oluşturulup kaydedilmiştir.

Sonuçların Değerlendirilmesi:

Bu raporda, Bölgesel Gezgin Satıcı Problemi (TSP) için geliştirilen Sezgisel (Heuristic), OR-Tools ve Genetik Algoritma yöntemlerinin 30 farklı topoloji üzerindeki performansları karşılaştırılmıştır. Karşılaştırma kriterleri olarak toplam yol maliyeti (mesafe), algoritma çalışma süreleri esas alınmıştır.

Genetik algoritmanın parametreleri olarak popülasyon sayısı 50, mutasyon oranı 0.1, elit oran 0.1, jenerasyon sayısı 20 olarak ayarlanmıştır.

● Toplam Mesafe Karşılaştırması



Şekil 1 Bölgesel TSP Yol Karşılaştırması

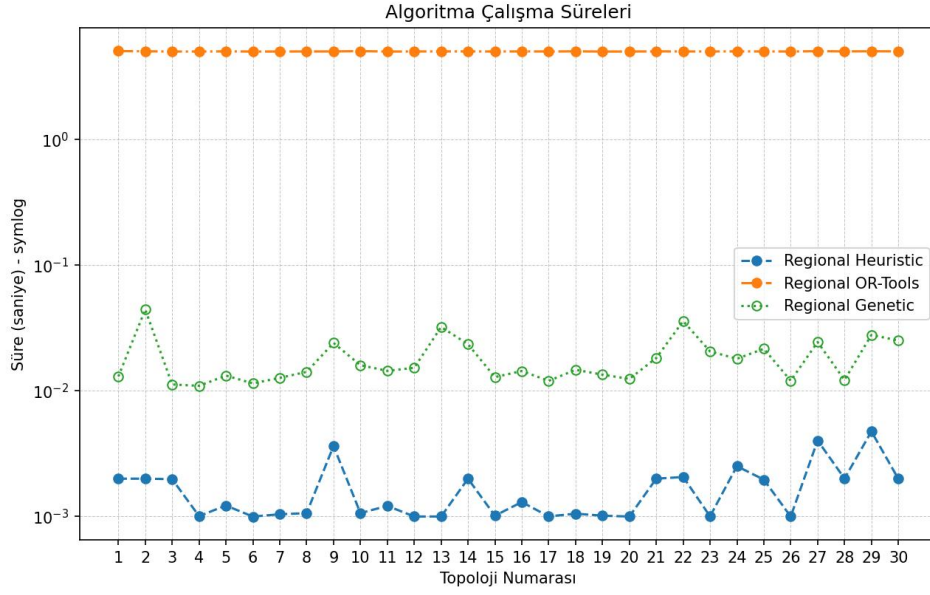
Şekil-1'deki çizgi grafiği incelendiğinde, 30 farklı topolojinin tamamında Regional OR-Tools (Turuncu çizgi) yönteminin en düşük maliyetli rotaları oluşturduğu gözlemlenmiştir. OR-Tools, Guided Local Search meta-sezgisel yöntemi ve bölgesel kısıtlamaları etkin kullanarak, her bölge içerisinde rotayı en çok kısaltacak düğümü seçmeyi başarmıştır.

Regional Heuristic (Mavi çizgi) ve Regional Genetic (Yeşil çizgi) algoritmaları ise genel olarak OR-Tools'u takip etmekle birlikte, çoğu topolojide daha yüksek maliyetli sonuçlar üretmiştir. Özellikle topoloji 6, 15 ve 23 gibi, OR-Tools ile diğer yöntemler arasındaki farkın açıldığı, ancak topoloji 2, 4, ve 9 gibi durumlarda üç algoritmanın da birbirine çok yakın sonuçlar verdiği görülmektedir.

Genetik algoritma, sezgisel yöntemle kıyasla bazı topolojilerde daha iyi sonuçlar (daha düşük mesafe) verdiği gözlemlenmiştir bu sorunu OR-Tools'un meta-sezgisel yöntemi 5 saniyelik bir limite çıkartılarak çözülmüştür.

● Algoritma Çalışma Süreleri:

Algoritmaların çalışma sürelerini minimize etmek amacıyla kod yapısında optimizasyon yapılmıştır. main.py içerisinde precompute_distances fonksiyonu ile, topolojideki tüm nodelar arasındaki en kısa yol mesafeleri, algoritmalar çalıştırılmadan önce bir kez hesaplanarak kullanılmıştır. Bu sayede 1 topolojide sadece sezgisel algoritmanın çalışma süresi 15, genetik algoritmanın çalışma süresi ise 10 dakikaları bulurken ön hesaplama sayesinde önemli ölçüde azalmıştır.



Şekil 2 Algoritma Çalışma Süreleri

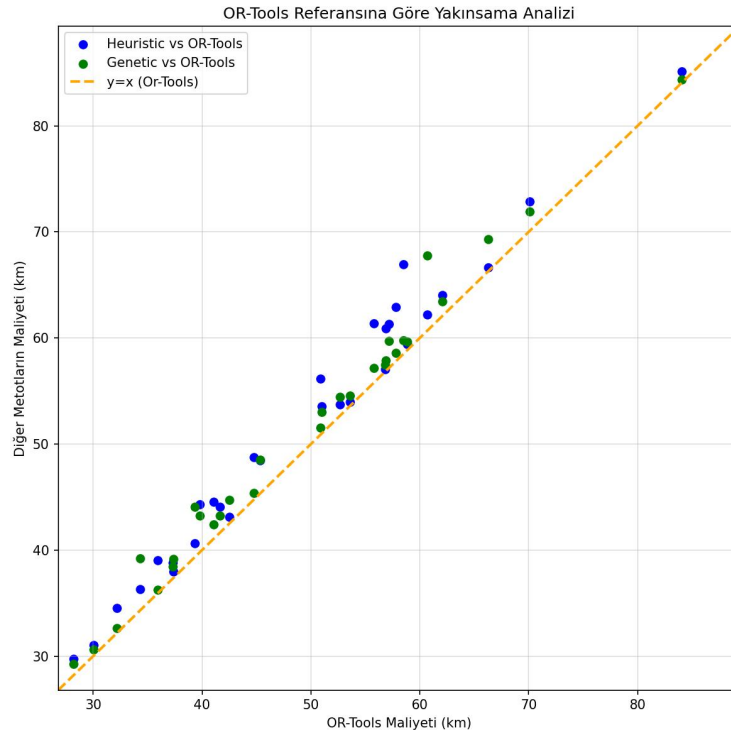
Şekil-2'te algoritmaların çalışma süreleri logaritmik ölçekte (symlog) karşılaştırılmıştır.

Regional Heuristic (Mavi): En hızlı çalışan yöntemdir. Süreler 10^{-3} ile 10^{-2} saniye bandında değişmekte olup, anlık çözüm gerektiren durumlar için en uygun yöntem olduğu söylenebilir.

Regional Genetic (Yeşil): Çalışma süresi 10^{-2} ile 10^{-1} saniye arasındadır. Sezgisel yöntemden daha yavaş olmasına rağmen kabul edilebilir sınırlar içerisindedir.

Regional OR-Tools (Turuncu): Grafikte sabit bir düz çizgi olarak görülmektedir. Bunun nedeni, koddaki zaman limitinin 5 saniyeye ayarlanmasıdır. Algoritma, en iyi sonucu bulmak için kendisine verilen 5 saniyelik süreyi sonuna kadar kullanmıştır. Bu süre, diğer yöntemlere göre çok daha uzun olsa da elde edilen yol kalitesi bu maliyeti haklı kabul edilebilir kılmaktadır.

● Yakınsama ve Dağılım Analizi



Şekil 3 OR-Tools Referansına Göre Yakınsama Analizi

Algoritmaların çözüm kalitesini daha net analiz etmek için Şekil-2'deki dağılım grafiği oluşturulmuştur. Bu grafikte x-ekseni en iyi sonucu veren OR-Tools Maliyeti, y-ekseni ise diğer metotların maliyetini göstermektedir. Turuncu kesikli çizgi ($y=x$) referans doğrusudur.

Mavi Noktalar (Heuristic vs OR-Tools): Noktaların neredeyse tamamının referans çizgisinin biraz üzerinde olduğu görülmektedir. Bu, sezgisel yöntemin OR-Tools'a göre daha kötü (daha uzun) yollar bulduğunu kanıtlamaktadır.

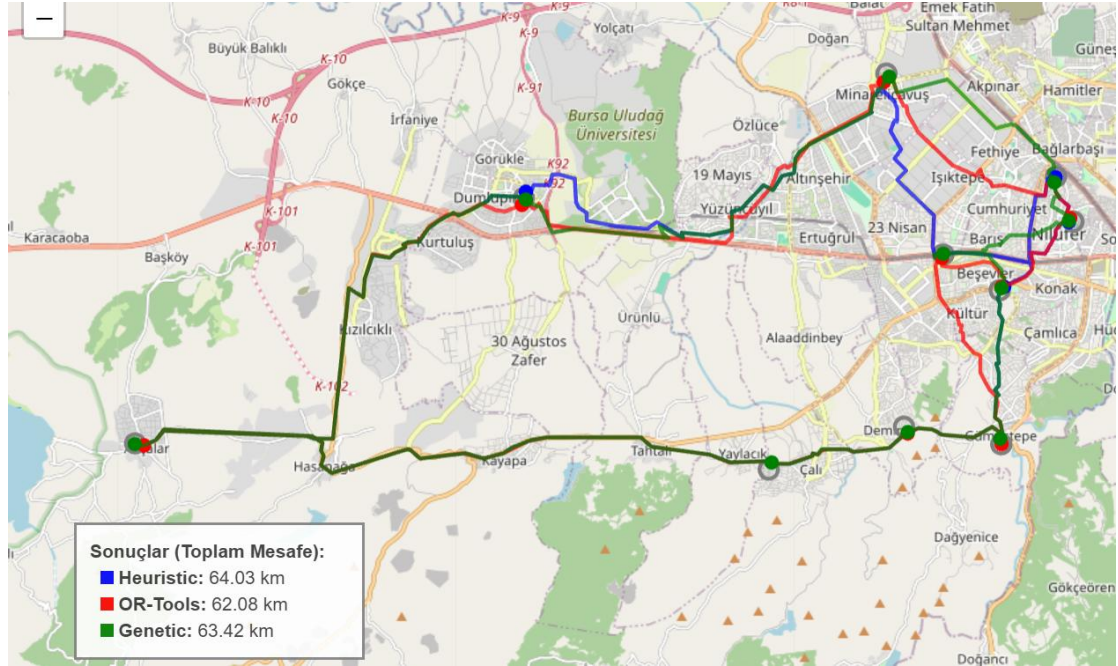
Yeşil Noktalar (Genetic vs OR-Tools): Genetik algoritma sonuçlarının da referans çizgisinin üzerinde yoğunlaştığı, ancak dağılımın genellikle sezgisel yöntemden daha iyi olduğu görülmektedir.

Bölgesel TSP probleminin, standart TSP'ye göre daha geniş bir arama uzayına (hem düğüm seçimi hem sıralama) sahip olması nedeniyle, Sezgisel ve Genetik algoritmaların global optimuma ulaşmakta zorlandığıdır. Ancak Genetik algoritma parametrelerini ayarlayarak Or-Tools sonuçlarına daha fazla yakınsayabilir. OR-Tools ise kendi içinde hibrit bir sistem barındırdığı için bu karmaşık yapıyı daha verimli yönetmiştir.

● Genel Değerlendirme

Yapılan analizler sonucunda; Bölgesel TSP probleminin çözümünde en kaliteli rotaların OR-Tools kütüphanesi ve Disjunction kısıtlaması ile elde edildiği belirlenmiştir. Sezgisel yöntem, hız açısından rakipsiz olsa da çözüm kalitesinde yetersiz kalabilmektedir.

Genetik algoritma ise parametre optimizasyonu (popülasyon sayısı, jenerasyon) ile geliştirilmeye açık bir yöntemdir. Bu değerlendirmede mevcut parametrelerle (50 popülasyon, 20 jenerasyon) sezgisel yöntemle benzer bir performans sergilemiştir. Gerçek hayat senaryolarında, eğer süre kısıtı yoksa OR-Tools, çok hızlı cevap verilmesi gereken durumlarda ise Sezgisel Yöntem veya Genetik Algoritma tercih edilmelidir.



Şekil 4 30. Topoloji karşılaştırmalı haritalama